

Лабораторная работа (Вариант 1)

C# + WPF + MySQL (Visual Studio Community 2026)

Автор: Каламбет В.Б.

подготовка к демонстрационному экзамену

Содержание

Шаг 1. Создайте рабочую структуру проекта	4
Шаг 2. Поднимите MySQL + phpMyAdmin через Docker Compose	4
Шаг 3. Создайте схему БД (модули 1–4)	5
Шаг 4. Подготовьте CSV из исходных XLSX (Через Excel)	7
Шаг 5. Импортируйте данные через UI phpMyAdmin	8
Шаг 5.1. Перенос raw-данных в боевые таблицы	8
Шаг 6. Выполните контрольные SQL-проверки	9
Шаг 7. Создайте WPF-проект в VS Community 2026	10
Шаг 8. Подключите NuGet-пакет MySQL	10
Шаг 9. Добавьте ресурсы и базовые файлы проекта	10
Шаг 9.1. Проверьте подключение к MySQL через код (Db.cs)	11
Шаг 10. Создайте модели (Models.cs)	12
Шаг 11. Создайте слой данных (DataService.cs)	13
Шаг 12. Создайте расчеты (Calculations.cs)	18
Шаг 13. Настройте MainWindow.xaml	19
Шаг 14. Настройте MainWindow.xaml.cs	21
Шаг 15. Создайте окно редактирования материала	23
Шаг 16. Создайте окно поставщиков	28
Шаг 17. Запустите приложение и проверьте сценарии	30
Шаг 18. Локальный git-коммит метода модуля 4	30
Шаг 19. Экспортируйте SQL-скрипт БД	30
Шаг 20. Сохраните ER-диаграмму БД в PDF	31
Шаг 21. Соберите исполняемый файл .exe (Release)	31
Шаг 22. Подготовьте финальный набор файлов для локального репозитория .	32

Результат лабораторной

После выполнения у вас будет приложение по модулям 1–4:

- MySQL-база данных в 3НФ;
 - ER-диаграмма БД в PDF;
 - импорт `xlsx` данных (через `csv`) в `phpMyAdmin`;
 - главная форма со списком материалов и расчетом стоимости партии;
 - форма добавления/редактирования материала;
 - окно поставщиков выбранного материала;
 - метод модуля 4 с локальным `git`-коммитом;
 - SQL-скрипт БД, исполняемый файл приложения и набор итоговых файлов для локального репозитория.
-

Шаг 1. Создайте рабочую структуру проекта

Что делаем

Создайте папку проекта и подкаталоги для кода и ресурсов.

Команды/код

```
cd C:\
mkdir demoexam_csharp
cd demoexam_csharp
mkdir resources
```

Скопируйте из архива в папку resources:

- Мозаика.png
 - Мозаика.ico
 - все файлы *_import.xlsx
-

Шаг 2. Поднимите MySQL + phpMyAdmin через Docker Compose

Что делаем

Создайте docker-compose.yml и запустите контейнеры. Альтернативно (без Docker) можно использовать XAMPP, Open Server Panel или локально установленный MySQL Server.

Команды/код

Создайте файл C:\demoexam_csharp\docker-compose.yml:

```
services:
  mysql:
    image: mysql:8.4
    container_name: demoexam_mysql
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: mosaic_demo
      MYSQL_USER: demo
      MYSQL_PASSWORD: demo
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

  phpmyadmin:
    image: phpmyadmin:latest
    container_name: demoexam_phpmyadmin
    restart: unless-stopped
```

```
environment:
  PMA_HOST: mysql
  PMA_PORT: 3306
  PMA_USER: root
  PMA_PASSWORD: root
ports:
  - "8081:80"
depends_on:
  - mysql

volumes:
  mysql_data:
```

Запуск:

```
cd C:\demoexam_csharp
docker compose up -d
docker compose ps
```

Шаг 3. Создайте схему БД (модули 1–4)

Что делаем

В phpMyAdmin создайте таблицы, связи, ограничения и служебные гав-таблицы для импорта.

Команды/код

1. Откройте phpMyAdmin:
 - при Docker: `http://localhost:8081`;
 - при XAMPP / Open Server Panel / локальном MySQL Server: адрес phpMyAdmin вашей локальной установки (часто `http://localhost/phpmyadmin`).
2. Войдите под пользователем с правами на создание таблиц (для Docker: `root / root`).
3. Если БД `mosaic_demo` уже есть — выберите ее.
4. Если БД `mosaic_demo` нет: вкладка **Базы данных** -> введите имя `mosaic_demo` -> выберите сравнение `utf8mb4_unicode_ci` -> нажмите **Создать** -> откройте созданную БД.
5. Откройте вкладку **SQL** и выполните скрипт:

```
USE mosaic_demo;
```

```
SET NAMES utf8mb4;
```

```
DROP TABLE IF EXISTS material_suppliers;
DROP TABLE IF EXISTS materials;
DROP TABLE IF EXISTS suppliers;
DROP TABLE IF EXISTS product_types;
DROP TABLE IF EXISTS material_types;
```

```

DROP TABLE IF EXISTS materials_import_raw;
DROP TABLE IF EXISTS material_suppliers_import_raw;

CREATE TABLE material_types (
    material_type_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    loss_percent DECIMAL(10,4) NOT NULL CHECK (loss_percent >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE product_types (
    product_type_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    coefficient DECIMAL(10,2) NOT NULL CHECK (coefficient > 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE suppliers (
    supplier_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL UNIQUE,
    supplier_type VARCHAR(50) NOT NULL,
    inn VARCHAR(12) NOT NULL UNIQUE,
    rating INT NOT NULL CHECK (rating >= 0),
    start_date DATE NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE materials (
    material_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL UNIQUE,
    material_type_id INT NOT NULL,
    unit_price DECIMAL(12,2) NOT NULL CHECK (unit_price >= 0),
    stock_quantity INT NOT NULL CHECK (stock_quantity >= 0),
    min_quantity INT NOT NULL CHECK (min_quantity >= 0),
    package_quantity INT NOT NULL CHECK (package_quantity > 0),
    unit_name VARCHAR(20) NOT NULL,
    CONSTRAINT fk_materials_type
        FOREIGN KEY (material_type_id) REFERENCES
material_types(material_type_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE material_suppliers (
    material_id INT NOT NULL,
    supplier_id INT NOT NULL,
    PRIMARY KEY (material_id, supplier_id),
    CONSTRAINT fk_ms_material
        FOREIGN KEY (material_id) REFERENCES materials(material_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_ms_supplier
        FOREIGN KEY (supplier_id) REFERENCES suppliers(supplier_id)
        ON UPDATE CASCADE ON DELETE RESTRICT

```

```

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE INDEX idx_materials_type ON materials(material_type_id);
CREATE INDEX idx_ms_supplier ON material_suppliers(supplier_id);

-- raw-таблицы только для удобного импорта csv с текстовыми названиями
CREATE TABLE materials_import_raw (
    name VARCHAR(150) NOT NULL,
    material_type_name VARCHAR(100) NOT NULL,
    unit_price DECIMAL(12,2) NOT NULL,
    stock_quantity INT NOT NULL,
    min_quantity INT NOT NULL,
    package_quantity INT NOT NULL,
    unit_name VARCHAR(20) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE material_suppliers_import_raw (
    material_name VARCHAR(150) NOT NULL,
    supplier_name VARCHAR(150) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Шаг 4 Подготовьте CSV из исходных XLSX (Через Excel)

Что делаем

Откройте каждый `xlsx` в Excel, сохраните как CSV UTF-8 и удалите строку заголовков.

Команды/код

Сохраните файлы:

- Material_type_import.csv
- Product_type_import.csv
- Suppliers_import.csv
- Materials_import.csv
- Material_suppliers_import.csv

В каждом CSV удалите первую строку (заголовки), чтобы файл начинался сразу с данных.

Важно:

1. Material_type_import.csv:
 - Во втором столбце должны быть значения вида 0.0012 (без знака %).
 - Если Excel показывает 0,12%, смените формат столбца на Общий или Числовой, затем замените запятую , на точку . (пример: 0,12 → 0.12).
2. Product_type_import.csv:
 - Во втором столбце замените , на . (пример: 8.59).
3. Suppliers_import.csv:

- Дата должна быть в формате YYYY-MM-DD (пример: 2015-12-20).
 - Проверьте, что ИНН не в экспоненциальной записи (9.432E+09), а обычным числом.
4. Materials_import.csv:
- Замените , на . в дробных значениях.
 - Выделите только целые колонки (Количество на складе, Минимальное количество, Количество в упаковке) и нажмите Уменьшить разрядность несколько раз, чтобы убрать хвосты ,00.
5. Material_suppliers_import.csv:
- Дополнительных числовых правок не требуется.

Шаг 5. Импортируйте данные через UI phpMyAdmin

Что делаем

Импортируйте CSV в таблицы в правильном порядке.

Команды/код

В phpMyAdmin:

1. Откройте таблицу material_types -> **Импорт** -> файл Material_type_import.csv -> поле Названия столбцов: name,loss_percent.
2. Откройте таблицу product_types -> **Импорт** -> файл Product_type_import.csv -> поле Названия столбцов: name,coefficient.
3. Откройте таблицу suppliers -> **Импорт** -> файл Suppliers_import.csv -> поле Названия столбцов: name,supplier_type,inn,rating,start_date.
4. Откройте таблицу materials_import_raw -> **Импорт** -> файл Materials_import.csv -> поле Названия столбцов: name,material_type_name,unit_price,stock_quantity,min_quantity,package_
5. Откройте таблицу material_suppliers_import_raw -> **Импорт** -> файл Material_suppliers_import.csv -> поле Названия столбцов: material_name,supplier_name.

Для каждого импорта обязательно выставьте:

- Формат = CSV.
- Разделитель полей = ; (для этой ЛР). Если ваш CSV с запятыми, поставьте ,.
- Значения полей обрамлены = ".
- Символ экранирования = " (оставьте значение по умолчанию в вашей версии).
- Разделитель строк = auto.
- Названия столбцов — заполните вручную, как указано в пунктах 1–5 выше.

Шаг 5.1. Перенос raw-данных в боевые таблицы

Что делаем

После импорта raw-таблиц перенесите данные в рабочие таблицы со связями.

Команды/код

Во вкладке **SQL** выполните:

```
USE mosaic_demo;
```

```
INSERT INTO materials (
    name, material_type_id, unit_price, stock_quantity, min_quantity,
    package_quantity, unit_name
)
SELECT
    r.name,
    mt.material_type_id,
    r.unit_price,
    r.stock_quantity,
    r.min_quantity,
    r.package_quantity,
    r.unit_name
FROM materials_import_raw r
JOIN material_types mt ON mt.name = r.material_type_name;

INSERT INTO material_suppliers (material_id, supplier_id)
SELECT
    m.material_id,
    s.supplier_id
FROM material_suppliers_import_raw r
JOIN materials m ON m.name = r.material_name
JOIN suppliers s ON s.name = r.supplier_name;
```

Шаг 6. Выполните контрольные SQL-проверки

Что делаем

Проверьте, что импорт соответствует целевым количествам.

Команды/код

```
USE mosaic_demo;
```

```
SELECT 'material_types' AS table_name, COUNT(*) AS cnt FROM material_types
UNION ALL
SELECT 'product_types', COUNT(*) FROM product_types
UNION ALL
SELECT 'suppliers', COUNT(*) FROM suppliers
UNION ALL
SELECT 'materials', COUNT(*) FROM materials
UNION ALL
SELECT 'material_suppliers', COUNT(*) FROM material_suppliers;
```

Шаг 7. Создайте WPF-проект в VS Community 2026

Что делаем

Создайте проект на шаблоне **Приложение WPF (.NET Framework)** (в некоторых сборках может называться **WPF Application (.NET Framework)**) и зафиксируйте 4.8.

Команды/код

1. Откройте Visual Studio Community 2026.
 2. Нажмите Создание проекта (Create a new project).
 3. Найдите шаблон: **Приложение WPF (.NET Framework)**.
 4. Имя проекта (Project name): MosaicMaterialsApp.
 5. Целевая платформа / Framework: **.NET Framework 4.8**.
 6. Расположение (Location): C:\demoexam_csharp.
-

Шаг 8. Подключите NuGet-пакет MySQL

Что делаем

Добавьте драйвер для подключения к MySQL.

Команды/код

Если окно Обзорщик решений не открыто: Вид -> Обзорщик решений.

1. В Обзорщик решений нажмите ПКМ по проекту MosaicMaterialsApp.
2. Выберите Управление пакетами NuGet....
3. Вкладка Обзор -> найдите MySql.Data.
4. Выберите версию 8.4.0 и нажмите Установить.

Альтернатива через консоль пакетов:

- Средства -> Диспетчер пакетов NuGet -> Консоль диспетчера пакетов
- или через поиск команд Visual Studio (Ctrl+Q) введите: Консоль диспетчера пакетов.

```
Install-Package MySql.Data -Version 8.4.0
```

Шаг 9. Добавьте ресурсы и базовые файлы проекта

Что делаем

Скопируйте изображения в проект и создайте служебные классы.

Команды/код

1. Если Обзорщик решений не открыт: Вид -> Обзорщик решений.
2. В Обзорщик решений: ПКМ по проекту MosaicMaterialsApp -> Добавить -> Создать папку -> имя resources.

3. В проводнике скопируйте Мозаика.png и Мозаика.ico в эту папку resources внутри папки проекта.
Пример: C:\demoexam_csharp\MosaicMaterialsApp\MosaicMaterialsApp\resources.
4. Если файлы не видны в Обозреватель решений, нажмите Показать все файлы, затем на Мозаика.png и Мозаика.ico нажмите ПКМ -> Включить в проект.
5. Для каждого файла откройте свойства (F4) и поставьте Действие при сборке = Resource.

Создайте файл Db.cs в Обозреватель решений:

1. ПКМ по проекту MosaicMaterialsApp.
2. Добавить -> Класс....
3. Имя файла: Db.cs.
4. Нажмите Добавить.

```
using MySql.Data.MySqlClient;
```

```
namespace MosaicMaterialsApp
{
    internal static class Db
    {
        public static readonly string ConnectionString =
"Server=127.0.0.1;Port=3306;Database=mosaic_demo;Uid=demo;Pwd=demo;Charset=utf8mb4;";

        public static MySqlConnection GetConnection()
        {
            return new MySqlConnection(ConnectionString);
        }
    }
}
```

Шаг 9.1. Проверьте подключение к MySQL через код (Db.cs)

Что делаем

Выполните быструю проверку подключения именно через метод Db.GetConnection().

Команды/код

1. Откройте файл App.xaml.cs (в Обозреватель решений).
2. Временно замените содержимое на:

```
using System.Windows;
```

```
namespace MosaicMaterialsApp
{
    public partial class App : Application
    {
        protected override void OnStartup(StartupEventArgs e)
```

```

    {
        base.OnStartup(e);

        try
        {
            using (var conn = Db.GetConnection())
            {
                conn.Open();
            }

            MessageBox.Show(
                "ОК: подключение к MySQL успешно.",
                "Проверка БД",
                MessageBoxButton.OK,
                MessageBoxImage.Information);
        }
        catch (System.Exception ex)
        {
            MessageBox.Show(
                "Ошибка подключения к MySQL:\n" + ex.Message,
                "Проверка БД",
                MessageBoxButton.OK,
                MessageBoxImage.Error);
        }
    }
}

```

3. Запустите проект: Отладка -> Начать отладку (F5).
4. После проверки верните App.xaml.cs в исходное состояние.

Шаг 10. Создайте модели (Models.cs)

Что делаем

Создайте модели для материалов, типов и поставщиков.

Команды/код

Создайте Models.cs в Обзорщике решений:

1. ПКМ по проекту MosaicMaterialsApp.
2. Добавить -> Класс....
3. Имя файла: Models.cs.
4. Нажмите Добавить.

```
using System;
```

```
namespace MosaicMaterialsApp
{
```

```

public class MaterialTypeItem
{
    public int MaterialTypeId { get; set; }
    public string Name { get; set; }
}

public class SupplierItem
{
    public string Name { get; set; }
    public int Rating { get; set; }
    public DateTime StartDate { get; set; }
}

public class MaterialItem
{
    public int MaterialId { get; set; }
    public int MaterialTypeId { get; set; }
    public string MaterialTypeName { get; set; }
    public string Name { get; set; }
    public decimal UnitPrice { get; set; }
    public int StockQuantity { get; set; }
    public int MinQuantity { get; set; }
    public int PackageQuantity { get; set; }
    public string UnitName { get; set; }

    public string Header => $"{MaterialTypeName} | {Name}";
    public string MinQuantityText => $"Минимальное количество: {MinQuantity}
{UnitName}";
    public string StockText => $"Количество на складе: {StockQuantity}
{UnitName}";
    public string PriceText => $"Цена: {UnitPrice:0.00} р / Единица
измерения: {UnitName}";
    public string BatchCostText =>
        $"Стоимость партии:
{Calculations.CalculateMinPurchaseCost(StockQuantity, MinQuantity,
PackageQuantity, UnitPrice):0.00} р";
}
}

```

Шаг 11. Создайте слой данных (DataService.cs)

Что делаем

Реализуйте чтение/запись материалов и поставщиков.

Команды/код

Создайте DataService.cs в Обозреватель решений:

1. ПКМ по проекту MosaicMaterialsApp.

2. Добавить -> Класс....

3. Имя файла: DataService.cs.

4. Нажмите Добавить.

```
using MySql.Data.MySqlClient;
using System.Collections.Generic;

namespace MosaicMaterialsApp
{
    internal static class DataService
    {
        public static List<MaterialTypeItem> GetMaterialTypes()
        {
            var result = new List<MaterialTypeItem>();
            using (var conn = Db.GetConnection())
            {
                conn.Open();
                var cmd = new MySqlCommand(
                    "SELECT material_type_id, name FROM material_types ORDER BY
name", conn);
                using (var reader = cmd.ExecuteReader())
                {
                    while (reader.Read())
                    {
                        result.Add(new MaterialTypeItem
                        {
                            MaterialTypeId = reader.GetInt32("material_type_id"),
                            Name = reader.GetString("name")
                        });
                    }
                }
            }
            return result;
        }

        public static List<MaterialItem> GetMaterials()
        {
            var result = new List<MaterialItem>();
            using (var conn = Db.GetConnection())
            {
                conn.Open();
                var cmd = new MySqlCommand(@"
SELECT
    m.material_id,
    m.material_type_id,
    mt.name AS material_type_name,
    m.name,
    m.unit_price,
    m.stock_quantity,
```

```

        m.min_quantity,
        m.package_quantity,
        m.unit_name
    FROM materials m
    JOIN material_types mt ON mt.material_type_id =
m.material_type_id
    ORDER BY m.material_id", conn);

    using (var reader = cmd.ExecuteReader())
    {
        while (reader.Read())
        {
            result.Add(new MaterialItem
            {
                MaterialId = reader.GetInt32("material_id"),
                MaterialTypeId = reader.GetInt32("material_type_id"),
                MaterialTypeName =
reader.GetString("material_type_name"),
                Name = reader.GetString("name"),
                UnitPrice = reader.GetDecimal("unit_price"),
                StockQuantity = reader.GetInt32("stock_quantity"),
                MinQuantity = reader.GetInt32("min_quantity"),
                PackageQuantity =
reader.GetInt32("package_quantity"),
                UnitName = reader.GetString("unit_name")
            });
        }
    }
    return result;
}

public static List<SupplierItem> GetSuppliersByMaterial(int materialId)
{
    var result = new List<SupplierItem>();
    using (var conn = Db.GetConnection())
    {
        conn.Open();
        var cmd = new MySqlCommand(@"
            SELECT s.name, s.rating, s.start_date
            FROM material_suppliers ms
            JOIN suppliers s ON s.supplier_id = ms.supplier_id
            WHERE ms.material_id = @materialId
            ORDER BY s.name", conn);
        cmd.Parameters.AddWithValue("@materialId", materialId);

        using (var reader = cmd.ExecuteReader())
        {
            while (reader.Read())

```

```

        {
            result.Add(new SupplierItem
            {
                Name = reader.GetString("name"),
                Rating = reader.GetInt32("rating"),
                StartDate = reader.GetDateTime("start_date")
            });
        }
    }
}

return result;
}

public static void InsertMaterial(MaterialItem item)
{
    using (var conn = Db.GetConnection())
    {
        conn.Open();
        var cmd = new MySqlCommand(@"
            INSERT INTO materials
            (name, material_type_id, unit_price, stock_quantity,
min_quantity, package_quantity, unit_name)
            VALUES
            (@name, @typeId, @price, @stock, @min, @pack, @unit)", conn);

        cmd.Parameters.AddWithValue("@name", item.Name);
        cmd.Parameters.AddWithValue("@typeId", item.MaterialTypeId);
        cmd.Parameters.AddWithValue("@price", item.UnitPrice);
        cmd.Parameters.AddWithValue("@stock", item.StockQuantity);
        cmd.Parameters.AddWithValue("@min", item.MinQuantity);
        cmd.Parameters.AddWithValue("@pack", item.PackageQuantity);
        cmd.Parameters.AddWithValue("@unit", item.UnitName);
        cmd.ExecuteNonQuery();
    }
}

public static void UpdateMaterial(MaterialItem item)
{
    using (var conn = Db.GetConnection())
    {
        conn.Open();
        var cmd = new MySqlCommand(@"
            UPDATE materials
            SET
            name = @name,
            material_type_id = @typeId,
            unit_price = @price,
            stock_quantity = @stock,
            min_quantity = @min,

```



```

        package_quantity = @pack,
        unit_name = @unit
    WHERE material_id = @id", conn);

    cmd.Parameters.AddWithValue("@id", item.MaterialId);
    cmd.Parameters.AddWithValue("@name", item.Name);
    cmd.Parameters.AddWithValue("@typeId", item.MaterialTypeId);
    cmd.Parameters.AddWithValue("@price", item.UnitPrice);
    cmd.Parameters.AddWithValue("@stock", item.StockQuantity);
    cmd.Parameters.AddWithValue("@min", item.MinQuantity);
    cmd.Parameters.AddWithValue("@pack", item.PackageQuantity);
    cmd.Parameters.AddWithValue("@unit", item.UnitName);
    cmd.ExecuteNonQuery();
}
}

public static bool TryGetCoefficients(int productTypeId, int
materialTypeId, out decimal productCoef, out decimal lossPercent)
{
    productCoef = 0;
    lossPercent = 0;

    using (var conn = Db.GetConnection())
    {
        conn.Open();
        var cmd = new MySqlCommand(@"
            SELECT
                (SELECT coefficient FROM product_types WHERE
product_type_id = @productTypeId) AS coefficient,
                (SELECT loss_percent FROM material_types WHERE
material_type_id = @materialTypeId) AS loss_percent", conn);

        cmd.Parameters.AddWithValue("@productTypeId", productTypeId);
        cmd.Parameters.AddWithValue("@materialTypeId", materialTypeId);

        using (var reader = cmd.ExecuteReader())
        {
            if (!reader.Read())
                return false;

            if (reader["coefficient"] == System.DBNull.Value ||
reader["loss_percent"] == System.DBNull.Value)
                return false;

            productCoef = (decimal)reader["coefficient"];
            lossPercent = (decimal)reader["loss_percent"];
            return true;
        }
    }
}

```

```

    }
}
}

```

Шаг 12. Создайте расчеты (Calculations.cs)

Что делаем

Добавьте оба обязательных метода расчета.

Команды/код

Создайте Calculations.cs в Обзоратель решений:

1. ПКМ по проекту MosaicMaterialsApp.
2. Добавить -> Класс....
3. Имя файла: Calculations.cs.
4. Нажмите Добавить.

```
using System;
```

```
namespace MosaicMaterialsApp
{
    internal static class Calculations
    {
        public static decimal CalculateMinPurchaseCost(int stockQuantity, int
minQuantity, int packageQuantity, decimal unitPrice)
        {
            if (stockQuantity >= minQuantity)
                return 0m;

            int deficit = minQuantity - stockQuantity;
            int packageCount = (int)Math.Ceiling((decimal)deficit /
packageQuantity);
            int purchaseAmount = packageCount * packageQuantity;

            var cost = purchaseAmount * unitPrice;
            if (cost < 0)
                return 0m;

            return Math.Round(cost, 2, MidpointRounding.AwayFromZero);
        }

        public static int CalculateProductsCount(int productTypeId, int
materialTypeId, int rawAmount, decimal param1, decimal param2)
        {
            if (productTypeId <= 0 || materialTypeId <= 0 || rawAmount < 0 ||
param1 <= 0 || param2 <= 0)
                return -1;
        }
    }
}
```

```

        decimal productCoef;
        decimal lossPercent;
        if (!DataService.TryGetCoefficients(productTypeId, materialTypeId,
out productCoef, out lossPercent))
            return -1;

        decimal rawPerUnit = param1 * param2 * productCoef;
        decimal rawPerUnitWithLoss = rawPerUnit * (1 + lossPercent);
        if (rawPerUnitWithLoss <= 0)
            return -1;

        return (int)Math.Floor(rawAmount / rawPerUnitWithLoss);
    }
}
}

```

Шаг 13. Настройте MainWindow.xaml

Что делаем

Сделайте главную форму с карточками материалов по стилю задания.

Команды/код

MainWindow.xaml уже создан автоматически при создании проекта. В Обозреватель решений откройте MainWindow.xaml и замените содержимое:

```

<Window x:Class="MosaicMaterialsApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Учет материалов"
        Height="820"
        Width="1200"
        Background="#FFFFFF"
        FontFamily="Comic Sans MS"
        Icon="resources/Мозаика.ico">
    <Window.Resources>
        <Style TargetType="Button">
            <Setter Property="Background" Value="#546F94"/>
            <Setter Property="Foreground" Value="White"/>
            <Setter Property="Margin" Value="5"/>
            <Setter Property="Padding" Value="10,6"/>
            <Setter Property="BorderThickness" Value="0"/>
        </Style>
    </Window.Resources>

    <DockPanel>
        <Border DockPanel.Dock="Top" Padding="12" Background="#FFFFFF">
            <Grid>
                <Grid.ColumnDefinitions>

```

```

        <ColumnDefinition Width="Auto"/>
        <ColumnDefinition Width="*/>
        <ColumnDefinition Width="Auto"/>
    </Grid.ColumnDefinitions>

    <Image Grid.Column="0" Source="resources/Мозаика.png" Width="90"
Height="90" Margin="0,0,12,0"/>
    <TextBlock Grid.Column="1" Text="Список материалов" FontSize="30"
FontWeight="Bold" VerticalAlignment="Center"/>
    <StackPanel Grid.Column="2" Orientation="Horizontal">
        <Button Content="Добавить материал"
Click="AddMaterial_Click"/>
        <Button Content="Обновить" Click="Refresh_Click"/>
        <Button Content="Проверка М4" Click="Module4Test_Click"/>
    </StackPanel>
</Grid>
</Border>

<ScrollView VerticalScrollBarVisibility="Auto">
    <ItemsControl ItemsSource="{Binding Materials}">
        <ItemsControl.ItemTemplate>
            <DataTemplate>
                <Border Margin="12,0,12,12" Padding="16"
Background="#ABCFCE" BorderBrush="#777777" BorderThickness="1">
                    <Grid>
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*/>
                            <ColumnDefinition Width="Auto"/>
                        </Grid.ColumnDefinitions>

                        <StackPanel Grid.Column="0">
                            <TextBlock Text="{Binding Header}"
FontSize="28" FontWeight="Bold"/>
                            <TextBlock Text="{Binding MinQuantityText}"
FontSize="20" Margin="0,8,0,0"/>
                            <TextBlock Text="{Binding StockText}"
FontSize="20"/>
                            <TextBlock Text="{Binding PriceText}"
FontSize="20"/>
                        </StackPanel>

                        <StackPanel Grid.Column="1"
HorizontalAlignment="Right">
                            <TextBlock Text="{Binding BatchCostText}"
FontSize="32" FontWeight="Bold"/>
                            <StackPanel Orientation="Horizontal"
HorizontalAlignment="Right" Margin="0,20,0,0">
                                <Button Content="Редактировать"
Tag="{Binding MaterialId}" Click="EditMaterial_Click"/>

```

```

                                <Button Content="Поставщики"
Tag="{Binding MaterialId}" Click="Suppliers_Click"/>
                                </StackPanel>
                            </StackPanel>
                        </Grid>
                    </Border>
                </DataTemplate>
            </ItemsControl.ItemTemplate>
        </ItemsControl>
    </ScrollViewer>
</DockPanel>
</Window>

```

Шаг 14. Настройте MainWindow.xaml.cs

Что делаем

Подключите загрузку данных, переходы и тест модуля 4.

Команды/код

В Обозреватель решений откройте MainWindow.xaml.cs и замените содержимое:

```

using System.Collections.ObjectModel;
using System.Linq;
using System.Text;
using System.Windows;
using System.Windows.Controls;

namespace MosaicMaterialsApp
{
    public partial class MainWindow : Window
    {
        public ObservableCollection<MaterialItem> Materials { get; } = new
ObservableCollection<MaterialItem>();

        public MainWindow()
        {
            InitializeComponent();
            DataContext = this;
            LoadMaterials();
        }

        private void LoadMaterials()
        {
            Materials.Clear();
            var items = DataService.GetMaterials();
            foreach (var item in items)
                Materials.Add(item);
        }
    }
}

```

```

private void AddMaterial_Click(object sender, RoutedEventArgs e)
{
    var window = new MaterialWindow(null);
    window.Owner = this;
    if (window.ShowDialog() == true)
        LoadMaterials();
}

private void EditMaterial_Click(object sender, RoutedEventArgs e)
{
    var button = sender as Button;
    if (button == null || button.Tag == null)
        return;

    int id;
    if (!int.TryParse(button.Tag.ToString(), out id))
        return;

    var item = Materials.FirstOrDefault(x => x.MaterialId == id);
    if (item == null)
        return;

    var window = new MaterialWindow(item);
    window.Owner = this;
    if (window.ShowDialog() == true)
        LoadMaterials();
}

private void Suppliers_Click(object sender, RoutedEventArgs e)
{
    var button = sender as Button;
    if (button == null || button.Tag == null)
        return;

    int id;
    if (!int.TryParse(button.Tag.ToString(), out id))
        return;

    var material = Materials.FirstOrDefault(x => x.MaterialId == id);
    if (material == null)
        return;

    var window = new SuppliersWindow(material.MaterialId, material.Name);
    window.Owner = this;
    window.ShowDialog();
}

private void Refresh_Click(object sender, RoutedEventArgs e)

```

```

    {
        LoadMaterials();
    }

    private void Module4Test_Click(object sender, RoutedEventArgs e)
    {
        var sb = new StringBuilder();
        sb.AppendLine("Тесты метода CalculateProductsCount:");
        sb.AppendLine($"1) {Calculations.CalculateProductsCount(1, 1, 1000, 2.5m, 1.5m)} (ожидаем 221)");
        sb.AppendLine($"2) {Calculations.CalculateProductsCount(2, 4, 5000, 1.2m, 2.1m)} (ожидаем 230)");
        sb.AppendLine($"3) {Calculations.CalculateProductsCount(4, 5, 12000, 3.0m, 0.8m)} (ожидаем 890)");
        sb.AppendLine($"4) {Calculations.CalculateProductsCount(99, 1, 100, 1.0m, 1.0m)} (ожидаем -1)");
        sb.AppendLine($"5) {Calculations.CalculateProductsCount(1, 1, 100, -1m, 2.0m)} (ожидаем -1)");

        MessageBox.Show(sb.ToString(), "Проверка модуля 4",
            MessageBoxButton.OK, MessageBoxImage.Information);
    }
}

```

Шаг 15. Создайте окно редактирования материала

Что делаем

Сделайте форму добавления/редактирования с валидациями.

Команды/код

Создайте окно MaterialWindow в Обзорщик решений:

1. ПКМ по проекту MosaicMaterialsApp.
2. Добавить -> Окно (WPF)... (или Добавить -> Новый элемент... -> Окно (WPF)).
3. Имя: MaterialWindow.xaml.
4. Нажмите Добавить.
5. Visual Studio автоматически создаст два файла: MaterialWindow.xaml и MaterialWindow.xaml.cs.

Замените содержимое MaterialWindow.xaml:

```

<Window x:Class="MosaicMaterialsApp.MaterialWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Добавление/редактирование материала"
        Height="520"
        Width="620"

```

```

        Background="#FFFFFF"
        FontFamily="Comic Sans MS"
        Icon="resources/Мозаика.ico"
        WindowStartupLocation="CenterOwner">
<Grid Margin="16">
    <Grid.RowDefinitions>
        <RowDefinition Height="*" />
        <RowDefinition Height="Auto" />
    </Grid.RowDefinitions>

    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="220" />
            <ColumnDefinition Width="*" />
        </Grid.ColumnDefinitions>

        <TextBlock Grid.Row="0" Grid.Column="0" Text="Наименование:"
Margin="0,8" />
        <TextBox x:Name="NameBox" Grid.Row="0" Grid.Column="1" Margin="0,8" />

        <TextBlock Grid.Row="1" Grid.Column="0" Text="Тип материала:"
Margin="0,8" />
        <ComboBox x:Name="MaterialTypeBox" Grid.Row="1" Grid.Column="1"
Margin="0,8" DisplayMemberPath="Name" SelectedValuePath="MaterialTypeId" />

        <TextBlock Grid.Row="2" Grid.Column="0" Text="Количество на складе:"
Margin="0,8" />
        <TextBox x:Name="StockBox" Grid.Row="2" Grid.Column="1" Margin="0,8" />
    >

    <TextBlock Grid.Row="3" Grid.Column="0" Text="Единица измерения:"
Margin="0,8" />
    <TextBox x:Name="UnitBox" Grid.Row="3" Grid.Column="1" Margin="0,8" />

    <TextBlock Grid.Row="4" Grid.Column="0" Text="Количество в упаковке:"
Margin="0,8" />
    <TextBox x:Name="PackageBox" Grid.Row="4" Grid.Column="1"
Margin="0,8" />

    <TextBlock Grid.Row="5" Grid.Column="0" Text="Минимальное

```



```

количество:" Margin="0,8"/>
    <TextBox x:Name="MinBox" Grid.Row="5" Grid.Column="1" Margin="0,8"/>

    <TextBlock Grid.Row="6" Grid.Column="0" Text="Цена единицы
материала:" Margin="0,8"/>
    <TextBox x:Name="PriceBox" Grid.Row="6" Grid.Column="1" Margin="0,8"/
>
    </Grid>

    <StackPanel Grid.Row="1" Orientation="Horizontal"
HorizontalAlignment="Right" Margin="0,16,0,0">
        <Button Content="Сохранить" Width="140" Margin="6"
Background="#546F94" Foreground="White" Click="Save_Click"/>
        <Button Content="Отмена" Width="120" Margin="6" Background="#546F94"
Foreground="White" Click="Cancel_Click"/>
    </StackPanel>
</Grid>
</Window>

```

Откройте MaterialWindow.xaml.cs (создан автоматически) и замените содержимое:

```

using System;
using System.Globalization;
using System.Linq;
using System.Windows;

namespace MosaicMaterialsApp
{
    public partial class MaterialWindow : Window
    {
        private readonly MaterialItem _editingItem;

        public MaterialWindow(MaterialItem editingItem)
        {
            InitializeComponent();
            _editingItem = editingItem;
            LoadMaterialTypes();
            if (_editingItem != null)
                FillForm();
        }

        private void LoadMaterialTypes()
        {
            MaterialTypeBox.ItemsSource = DataService.GetMaterialTypes();
        }

        private void FillForm()
        {
            NameBox.Text = _editingItem.Name;
            MaterialTypeBox.SelectedValue = _editingItem.MaterialTypeId;
        }
    }
}

```

```

        StockBox.Text = _editingItem.StockQuantity.ToString();
        UnitBox.Text = _editingItem.UnitName;
        PackageBox.Text = _editingItem.PackageQuantity.ToString();
        MinBox.Text = _editingItem.MinQuantity.ToString();
        PriceBox.Text = _editingItem.UnitPrice.ToString("0.00",
CultureInfo.InvariantCulture);
    }

    private bool TryParseDecimal(string text, out decimal value)
    {
        return decimal.TryParse(
            text.Replace(',', '.'),
            NumberStyles.Number,
            CultureInfo.InvariantCulture,
            out value);
    }

    private void Save_Click(object sender, RoutedEventArgs e)
    {
        if (string.IsNullOrEmpty(NameBox.Text) ||
string.IsNullOrEmpty(UnitBox.Text))
        {
            MessageBox.Show("Заполните наименование и единицу измерения.",
                "Предупреждение", MessageBoxButton.OK,
MessageBoxImage.Warning);
            return;
        }

        if (MaterialTypeBox.SelectedItem == null)
        {
            MessageBox.Show("Выберите тип материала.",
                "Предупреждение", MessageBoxButton.OK,
MessageBoxImage.Warning);
            return;
        }

        int stock;
        int packageQty;
        int minQty;
        decimal price;

        if (!int.TryParse(StockBox.Text, out stock) ||
            !int.TryParse(PackageBox.Text, out packageQty) ||
            !int.TryParse(MinBox.Text, out minQty) ||
            !TryParseDecimal(PriceBox.Text, out price))
        {
            MessageBox.Show("Проверьте числовые поля.",
                "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
            return;
        }
    }

```

```

    }

    if (stock < 0 || packageQty <= 0 || minQty < 0 || price < 0)
    {
        MessageBox.Show("Количество не может быть отрицательным, упаковка  
должна быть > 0, цена не может быть отрицательной.",  
            "Ошибка ввода", MessageBoxButton.OK, MessageBoxImage.Error);
        return;
    }

    if (decimal.Round(price, 2) != price)
    {
        MessageBox.Show("Цена должна содержать не более 2 знаков после  
запятой.",  
            "Предупреждение", MessageBoxButton.OK,  
            MessageBoxImage.Warning);
        return;
    }

    var type = (MaterialTypeItem)MaterialTypeBox.SelectedItem;
    var item = new MaterialItem
    {
        MaterialId = _editingItem == null ? 0 : _editingItem.MaterialId,
        MaterialTypeId = type.MaterialTypeId,
        MaterialTypeName = type.Name,
        Name = NameBox.Text.Trim(),
        StockQuantity = stock,
        UnitName = UnitBox.Text.Trim(),
        PackageQuantity = packageQty,
        MinQuantity = minQty,
        UnitPrice = price
    };

    try
    {
        if (_editingItem == null)
            DataService.InsertMaterial(item);
        else
            DataService.UpdateMaterial(item);

        DialogResult = true;
        Close();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Не удалось сохранить материал: " + ex.Message,  
            "Ошибка", MessageBoxButton.OK, MessageBoxImage.Error);
    }
}

```

```

        private void Cancel_Click(object sender, RoutedEventArgs e)
        {
            DialogResult = false;
            Close();
        }
    }
}

```

Шаг 16. Создайте окно поставщиков

Что делаем

Реализуйте отдельное окно списка поставщиков выбранного материала.

Команды/код

Создайте окно SuppliersWindow в Обозреватель решений:

1. ПКМ по проекту MosaicMaterialsApp.
2. Добавить -> Окно (WPF)... (или Добавить -> Новый элемент... -> Окно (WPF)).
3. Имя: SuppliersWindow.xaml.
4. Нажмите Добавить.
5. Visual Studio автоматически создаст два файла: SuppliersWindow.xaml и SuppliersWindow.xaml.cs.

Замените содержимое SuppliersWindow.xaml:

```

<Window x:Class="MosaicMaterialsApp.SuppliersWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Поставщики материала"
    Height="420"
    Width="780"
    Background="#FFFFFF"
    FontFamily="Comic Sans MS"
    Icon="resources/Мозаика.ico"
    WindowStartupLocation="CenterOwner">
    <Grid Margin="12">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <TextBlock x:Name="HeaderText" FontSize="22" FontWeight="Bold"
Margin="0,0,0,10"/>

        <DataGrid x:Name="SuppliersGrid"
            Grid.Row="1"

```

```

        AutoGenerateColumns="False"
        IsReadOnly="True"
        HeadersVisibility="Column">
        <DataGrid.Columns>
            <DataGridTextColumn Header="Наименование поставщика"
Binding="{Binding Name}" Width="*" />
            <DataGridTextColumn Header="Рейтинг" Binding="{Binding Rating}"
Width="120" />
            <DataGridTextColumn Header="Дата начала работы" Binding="{Binding
StartDate, StringFormat=dd.MM.yyyy}" Width="180" />
        </DataGrid.Columns>
    </DataGrid>

    <Button Grid.Row="2"
        Content="Назад"
        Width="120"
        HorizontalAlignment="Right"
        Margin="0,10,0,0"
        Background="#546F94"
        Foreground="White"
        Click="Back_Click" />
</Grid>
</Window>

```

Откройте SuppliersWindow.xaml.cs (создан автоматически) и замените содержимое:

```

using System.Windows;

namespace MosaicMaterialsApp
{
    public partial class SuppliersWindow : Window
    {
        public SuppliersWindow(int materialId, string materialName)
        {
            InitializeComponent();
            HeaderText.Text = "Поставщики материала: " + materialName;
            SuppliersGrid.ItemsSource =
DataService.GetSuppliersByMaterial(materialId);
        }

        private void Back_Click(object sender, RoutedEventArgs e)
        {
            Close();
        }
    }
}

```

Шаг 17. Запустите приложение и проверьте сценарии

Что делаем

Проверьте все обязательные сценарии.

Команды/код

Запуск в Visual Studio:

- Сборка -> Пересобрать решение
- Отладка -> Начать отладку (F5)

Проверить:

1. Загрузка карточек материалов.
 2. Корректный расчет «Стоимость партии».
 3. Добавление материала.
 4. Редактирование материала.
 5. Показ поставщиков.
 6. Кнопка Проверка М4 дает ожидаемые значения:
 - 221
 - 230
 - 890
 - -1
 - -1
-

Шаг 18. Локальный git-коммит метода модуля 4

Что делаем

Зафиксируйте локальный коммит проекта после реализации метода модуля 4.

Команды/код

```
cd C:\demoexam_csharp\MosaicMaterialsApp
git init
git add .
git commit -m "Добавлен метод CalculateProductsCount (модуль 4)"
```

Шаг 19. Экспортируйте SQL-скрипт БД

Что делаем

Выгрузите итоговый SQL для сдачи.

Команды/код

1. В phpMyAdmin выберите mosaic_demo.
2. Откройте вкладку **Экспорт**.

3. В блоке Метод экспорта выберите Обычный - отображать все возможные настройки.
 4. Проверьте:
 - Формат = SQL;
 - в блоке Таблицы включены Структура и Данные;
 - в блоке Параметры создания объектов при необходимости включены Добавить выражение DROP TABLE / VIEW / PROCEDURE / FUNCTION / EVENT / TRIGGER и IF NOT EXISTS;
 - в блоке Параметры создания данных оператор = INSERT.
 5. В блоке Вывод оставьте Сохранить вывод в файл.
 6. Нажмите **Экспорт** и сохраните файл как mosaic_demo.sql.
-

Шаг 20. Сохраните ER-диаграмму БД в PDF

Что делаем

Получите ER-диаграмму средствами phpMyAdmin (требование модуля 1).

Команды/код

1. В phpMyAdmin откройте БД mosaic_demo.
 2. В верхнем меню БД выберите **Ещё** -> **Дизайнер**.
 3. В дизайнера нажмите **Экспорт схемы**.
 4. Выберите формат PDF.
 5. Сохраните файл как mosaic_demo_er.pdf.
-

Шаг 21. Соберите исполняемый файл .exe (Release)

Что делаем

Соберите WPF-приложение в Release.

Команды/код

1. В Visual Studio переключите конфигурацию на Release (верхняя панель рядом с Debug/Any CPU).
2. Меню Сборка -> Собрать решение.
3. Проверьте папку сборки:
 - C:\demoexam_csharp\MosaicMaterialsApp\MosaicMaterialsApp\bin\Release\
4. В этой папке должен быть MosaicMaterialsApp.exe и зависимые файлы (.dll, .config).
5. Если используете другой путь проекта, берите аналогичный путь ... \bin\Release\.

Важно: для WPF (.NET Framework 4.8) обычно это не один отдельный файл, рабочим набором считается папка bin\Release целиком.

Шаг 22. Подготовьте финальный набор файлов для локального репозитория

Что делаем

Соберите итоговые файлы строго по формулировке задания.

Команды/код

Подготовьте в папке проекта:

- папка исходного кода проекта (структура с файлами, не архив);
- папка bin\Release целиком (включая MosaicMaterialsApp.exe и зависимости);
- mosaic_demo.sql (скопируйте экспортированный файл в папку проекта);
- mosaic_demo_er.pdf (скопируйте экспортированный файл в папку проекта).

Зафиксируйте итог в локальном git-репозитории:

```
cd C:\demoexam_csharp\MosaicMaterialsApp
git add .
git status
git commit -m "Финальная версия проекта (C#)"
```
